

LAPORAN PRAKTIKUM

IMPLEMENTASI HIGH AVAILABILITY SERVER DENGAN LOAD BALANCING

Disusun untuk laporan kemajuan tahap tiga agar memenuhi tugas proyek akhir kelompok
Mata Kuliah Jaringan Komputer



KELOMPOK 7

- | | | |
|---|----------------------------|--------------|
| 1 | MUHAMMAD ADIB HARYADI | (2401020076) |
| 2 | DHIYA ZARIFA PUTRI MARZUKI | (2401020075) |
| 3 | NISRINA RETNOSARI | (2401020060) |
| 4 | GIFFARI ZAKKA WALI ANDRY | (2401020088) |
| 5 | MUHAMMAD FATHIR | (2401020082) |

Dosen Pengampu:

FERDI CAHYADI, S. KOM., M. Cs.

TEKNIK INFORMATIKA
JURUSAN TEKNIK ELEKTRO DAN INFORMATIKA
FAKULTAS TEKNIK DAN TEKNOLOGI KEMARITIMAN
UNIVERSITAS MARITIM RAJA ALI HAJI

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya sehingga Laporan Kemajuan tahap tiga Proyek Implementasi High Availability Server dengan Load Balancing ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi perkembangan pelaksanaan Project Based Learning (PjBL) pada mata kuliah Jaringan Komputer.

Laporan Kemajuan tahap tiga ini berisi hasil implementasi tahap awal yang meliputi persiapan lingkungan kerja, pembuatan virtual machine, konfigurasi jaringan, serta integrasi perangkat pada simulator GNS3. Tahapan tersebut merupakan dasar yang penting sebelum dilakukan implementasi layanan web, konfigurasi HAProxy, serta pengujian mekanisme load balancing dan failover pada tahap selanjutnya.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Jaringan Komputer atas arahan yang diberikan selama proses pembelajaran, serta kepada seluruh anggota Kelompok 7 yang telah bekerja sama dalam proses perancangan dan implementasi proyek ini.

Penulis menyadari bahwa laporan ini masih memiliki keterbatasan dan masih dapat disempurnakan. Oleh karena itu, kritik dan saran yang bersifat membangun sangat diharapkan sebagai bahan evaluasi untuk penyusunan laporan pada tahap berikutnya.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca serta menjadi dokumentasi perkembangan implementasi sistem High Availability Server yang sedang dikembangkan.

Tanjungpinang, 16 Juni 2026

Kelompok 7

DAFTAR ISI

[_Toc233468028](#)

DAFTAR ISI.....	iii
DAFTAR TABLE.....	iv
DAFTAR GAMBAR.....	v
BAB I PENDAHULUAN.....	0
1.1 Latar Belakang	0
1.2 Rumusan Masalah	1
1.3 Tujuan.....	1
1.4 Manfaat.....	2
BAB II ANALISIS KEBUTUHAN.....	3
2.1 Kebutuhan Fungsional.....	3
2.2 Kebutuhan Non-Fungsional	3
2.3 Kebutuhan Perangkat Lunak dan Keras	4
2.3.1 Perangkat Lunak	4
2.3.2 Spesifikasi Hardware Minimum yang Direkomendasikan	4
BAB III DESAIN TOPOLOGI DAN IP PLAN	5
3.1 Topologi Jaringan.....	5
3.2 Perancangan Skema Pengalamatan IP dan Routing.....	7
3.2.1 Proses Kalkulasi VLSM.....	7
3.2.2 Pembagian Subnet (VLSM).....	7
3.2.3 2 IP Address Plan per Perangkat.....	8
3.2.4 Routing Table (Static Routes).....	8
3.3 Alur Komunikasi Sistem	9
BAB IV ANALISIS KEBUTUHAN	10
4.1 Persiapan Lingkungan Implementasi	10
4.2 Pembuatan Virtual Machine Ubuntu Server	10
4.3 Konfigurasi Jaringan Virtual Machine.....	11
4.4 Integrasi Virtual Machine dengan GNS3	13
4.5 Hasil Implementasi Tahap Awal	13
4.6 Instalasi dan Konfigurasi Apache2	15
4.7 Konfigurasi HAProxy	17
4.8 Pengujian Load Balancing	18
4.9 Pengujian Failover.....	19
BAB V PENUTUP	21
DAFTAR PUSTAKA	22

DAFTAR TABLE

Table 1 Kebutuhan Non-Fungsional Sistem	3
Table 2 Kebutuhan Perangkat Lunak.....	4
Table 3 Spesifikasi Hardware Minimum	4
Table 4 Komponen-komponen dalam topologi.	5
Table 5 Pembagian Subnet menggunakan VLSM	7
Table 6 IP Address Plan per Perangkat.....	8
Table 7 Routing Table pada router 1	8
Table 8 Fungsi virtual machine yang digunakan.	10

DAFTAR GAMBAR

Gambar 1 Alur trafik normal.	6
Gambar 2 Alur trafik saat failover	6
Gambar 3 Topologi Jaringan High Availability	6
Gambar 4 Konfigurasi IP Address pada HAProxy	11
Gambar 5 Konfigurasi alamat IP pada Web Server 1.....	12
Gambar 6 Konfigurasi alamat IP pada Web Server 2.....	12
Gambar 7 Topologi implementasi jaringan pada GNS3.....	13
Gambar 8 Hasil traceroute dari PC1 menuju HAProxy	14
Gambar 9 Routing table pada Router R1	14
Gambar 10 Pengujian konektivitas dari PC1 ke seluruh perangkat.....	15
Gambar 11 Status layanan Apache2 pada Web Server 1.....	16
Gambar 12 Status layanan Apache2 pada Web Server 2.....	17
Gambar 13 Konfigurasi file haproxy.cfg.....	17
Gambar 14 Status layanan HAProxy beserta health check backend	18
Gambar 15 Hasil pengujian Round Robin menggunakan perintah curl.....	19
Gambar 16 Hasil pengujian failover ketika salah satu backend server tidak aktif.	20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang pesat telah mendorong kebutuhan akan layanan web yang handal dan terus-menerus tersedia. Organisasi modern sangat bergantung pada infrastruktur web untuk menjalankan berbagai layanan bisnis, mulai dari transaksi elektronik, sistem informasi, hingga komunikasi antar pengguna. Dalam kondisi tersebut, ketersediaan layanan (*availability*) menjadi salah satu aspek kritis yang harus dijamin.

Arsitektur web konvensional yang hanya menggunakan satu server (*single server*) mengandung kelemahan mendasar, yaitu adanya *Single Point of Failure* (SPOF). Kondisi ini berarti bahwa jika server tunggal mengalami gangguan, baik akibat kegagalan perangkat keras, lonjakan trafik, maupun pemeliharaan sistem, maka seluruh layanan akan berhenti dan tidak dapat diakses oleh pengguna. Dampak dari kondisi ini dapat berupa kerugian finansial, menurunnya kepercayaan pengguna, serta terganggunya operasional bisnis secara keseluruhan.

Untuk mengatasi permasalahan tersebut, konsep *High Availability* (HA) hadir sebagai solusi arsitektur yang dirancang untuk memastikan layanan tetap berjalan meskipun salah satu komponen infrastruktur mengalami kegagalan. Implementasi *High Availability* umumnya dilengkapi dengan mekanisme *Load Balancing*, yang berfungsi mendistribusikan beban trafik secara merata ke beberapa server *backend* sehingga tidak ada satu server pun yang mengalami kelebihan beban.

HAProxy (*High Availability Proxy*) merupakan salah satu perangkat lunak load balancer open-source yang paling banyak digunakan di industri. HAProxy mampu menangani jutaan koneksi secara bersamaan dengan latensi rendah, mendukung berbagai algoritma penyeimbangan beban seperti *Round Robin* dan *Least Connection*, serta dilengkapi dengan fitur *health check* yang secara otomatis mendeteksi server *backend* yang mengalami kegagalan dan mengalihkan trafik ke server yang masih aktif (*failover* otomatis). (Tanenbaum & Wetherall, 2011).

Dalam proyek PjBL (*Project-Based Learning*) ini, Kelompok 7 akan mengimplementasikan infrastruktur *High Availability Server* menggunakan HAProxy sebagai *load balancer* dengan dua *web server backend* berbasis Apache2 yang berjalan di atas Ubuntu Server 22.04 LTS. Simulasi jaringan dilakukan menggunakan GNS3 untuk

mensimulasikan lingkungan jaringan yang realistis, sehingga pengujian failover dan distribusi trafik dapat dilakukan secara komprehensif. (HAProxy Technologies, 2024)

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam proyek ini adalah sebagai berikut:

1. Bagaimana merancang dan mengimplementasikan infrastruktur jaringan *High Availability* menggunakan HAProxy sebagai load balancer dengan dua web server backend dalam lingkungan simulasi GNS3?
2. Bagaimana mekanisme distribusi trafik (*load balancing*) dengan algoritma Round Robin dapat diterapkan agar beban permintaan dari klien terbagi secara merata ke seluruh server backend?
3. Bagaimana sistem dapat mendeteksi kegagalan server backend secara otomatis melalui mekanisme health check dan melakukan failover ke server yang masih aktif tanpa interupsi layanan yang signifikan?
4. Seberapa besar peningkatan availability layanan web yang dicapai dengan implementasi High Availability dibandingkan dengan arsitektur single server konvensional?

1.3 Tujuan

Tujuan Umum:

Merancang, mengimplementasikan, dan menguji infrastruktur High Availability Server dengan Load Balancing menggunakan HAProxy dalam lingkungan simulasi GNS3 sebagai solusi untuk mengatasi masalah Single Point of Failure pada layanan web.

Tujuan Khusus:

1. Merancang topologi jaringan dan skema pengalamatan IP (IP addressing) yang sesuai untuk infrastruktur High Availability menggunakan metode VLSM (Variable Length Subnet Mask).
2. Mengimplementasikan konfigurasi routing statis pada router GNS3 untuk memastikan konektivitas antar segmen jaringan.
3. Mengonfigurasi HAProxy sebagai load balancer dengan algoritma Round Robin dan mengaktifkan fitur health check untuk pemantauan status server backend secara real-time.

4. Melakukan serangkaian pengujian meliputi distribusi trafik, failover server, dan availability testing untuk mengukur performa sistem secara kuantitatif.

1.4 Manfaat

Manfaat Teoritis:

1. Memberikan pemahaman mendalam mengenai konsep High Availability, Load Balancing, dan Failover dalam konteks infrastruktur jaringan modern.
2. Menerapkan pengetahuan routing jaringan (static routing) dalam simulasi jaringan nyata menggunakan GNS3.
3. Memberikan referensi implementasi HAProxy beserta analisis performa dalam lingkungan akademik.

Manfaat Praktis:

1. Menghasilkan dokumentasi teknis yang dapat dijadikan panduan implementasi High Availability Server di lingkungan nyata.
2. Meningkatkan kompetensi teknis anggota kelompok dalam bidang administrasi jaringan, konfigurasi server Linux, dan pengujian infrastruktur.
3. Memberikan gambaran konkret kepada institusi mengenai penerapan teknologi HA dalam meningkatkan keandalan layanan web organisasi.

BAB II

ANALISIS KEBUTUHAN

2.1 Kebutuhan Fungsional

Sistem yang akan dibangun harus mampu memenuhi kebutuhan fungsional berikut:

1. Sistem harus dapat mendistribusikan trafik HTTP dari klien secara merata ke dua web server backend (Web Server 1 dan Web Server 2) menggunakan algoritma Round Robin.
2. Sistem harus dapat mendeteksi kondisi server backend yang mengalami kegagalan secara otomatis melalui mekanisme health check yang dikonfigurasi pada HAProxy.
3. Sistem harus tetap dapat melayani permintaan klien (memberikan respons HTTP 200 OK) meskipun salah satu server backend dalam kondisi tidak aktif (down), dengan mengalihkan trafik ke server backend yang masih aktif (failover otomatis).
4. Router harus mampu meneruskan paket data antar segmen jaringan (client network, frontend network, dan backend network) menggunakan konfigurasi routing statis.
5. Setiap web server harus menampilkan halaman web yang mengidentifikasi dirinya ("Web Server 1" atau "Web Server 2") sehingga distribusi trafik dapat diverifikasi secara visual.

2.2 Kebutuhan Non-Fungsional

Table 1 Kebutuhan Non-Fungsional Sistem

No	Parameter	Target	Keterangan
1	Availability	$\geq 99\%$	Uptime layanan web saat pengujian berlangsung
2	Failover Time	≤ 10 detik	Waktu deteksi kegagalan server hingga trafik dialihkan
3	Response Time	< 500 ms	Tidak meningkat signifikan saat proses failover aktif
4	Reproducibility	100%	Konfigurasi dapat direproduksi di lingkungan GNS3 lain
5	Scalability	Modular	Arsitektur mendukung penambahan server backend

2.3 Kebutuhan Perangkat Lunak dan Keras

2.3.1 Perangkat Lunak

Table 2 Kebutuhan Perangkat Lunak

No	Perangkat Lunak	Versi	Fungsi
1	GNS3	2.2.x	Simulasi topologi jaringan (router, switch, cloud)
2	Ubuntu Server	22.04 LTS	OS untuk HAProxy, Web Server 1, dan Web Server 2
3	HAProxy	2.4.x	Load balancer dan health check backend
4	Apache2 / Nginx	2.4.x	Web server pada WS1 dan WS2
5	VirtualBox	7.x	Hypervisor untuk menjalankan VM Ubuntu
6	Draw.io	Latest	Perancangan diagram topologi jaringan
7	curl / wget	Bawaan Ubuntu	Pengujian konektivitas dan request HTTP

2.3.2 Spesifikasi Hardware Minimum yang Direkomendasikan

Table 3 Spesifikasi Hardware Minimum

Komponen	Spesifikasi Minimum	Rekomendasi
Prosesor	Intel Core i5 (4 core)	Intel Core i7 / AMD Ryzen 5 (6+ core)
RAM	8 GB	16 GB (untuk menjalankan 3–4 VM secara bersamaan)
Storage	50 GB HDD	100 GB SSD (untuk performa VM yang lebih baik)
OS Host	Windows 10 64-bit	Windows 11 / Ubuntu 22.04 Desktop
Network Adapter	1 NIC	Gigabit Ethernet

BAB III

DESAIN TOPOLOGI DAN IP PLAN

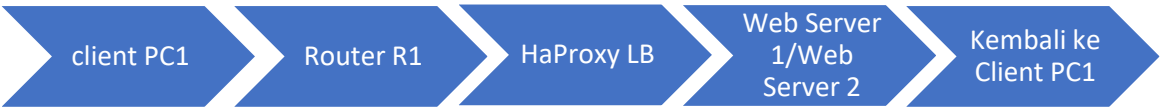
3.1 Topologi Jaringan

Infrastruktur High Availability yang dirancang pada proyek ini terdiri dari beberapa komponen utama yang saling terhubung dalam dua segmen jaringan terpisah. Pemisahan jaringan bertujuan untuk meningkatkan keamanan dan mengisolasi trafik frontend (antara klien dan load balancer) dari trafik backend (antara load balancer dan web server).

Table 4 Komponen-komponen dalam topologi.

No.	Perangkat	Fungsi
1.	Internet/Cloud (NAT)	Mewakili jaringan luar atau sumber trafik dari luar.
2.	Router R1	Gateway utama yang menghubungkan jaringan klien dengan HAProxy dan berfungsi sebagai penerus paket antar subnet (IP forwarding).
3.	Switch SW1	Switch utama pada jaringan frontend yang menghubungkan Router R1, HAProxy, dan Client PC.
4.	HAProxy Load Balancer (LB)	Komponen inti sistem HA; menerima semua request dari klien, mendistribusikannya ke WS1/WS2 menggunakan Round Robin, dan melakukan health check secara berkala.
5.	Switch SW2	Switch backend yang mengisolasi jaringan antara HAProxy dan kedua web server.
6.	Web Server 1 (WS1)	Server backend pertama yang menjalankan Apache2/Nginx.
7.	Web Server 2 (WS2)	Server backend kedua yang menjalankan Apache2/Nginx.
8.	Client PC1	Mesin klien untuk pengujian distribusi trafik dan failover.

Alur Trafik Normal:

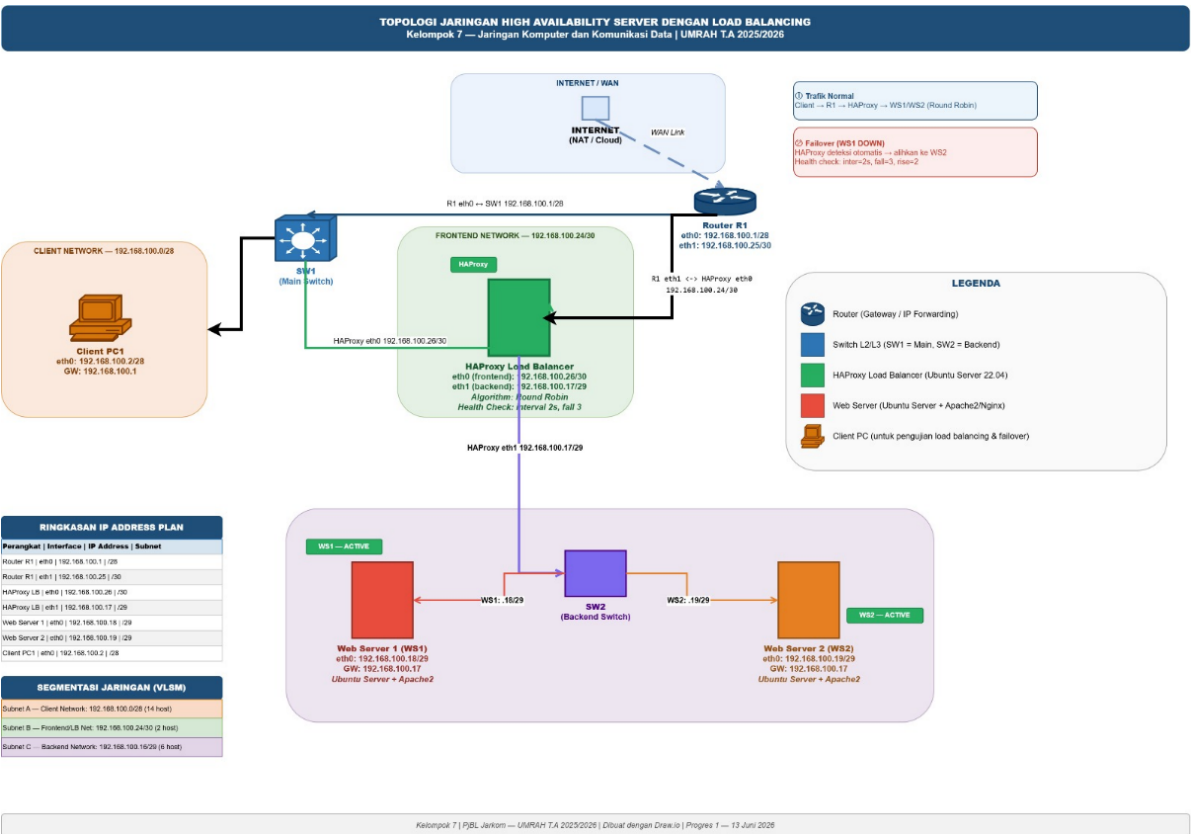


Gambar 1 Alur trafik normal.

Alur Trafik Saat Failover:



Gambar 2 Alur trafik saat failover



Gambar 3 Topologi Jaringan High Availability

3.2 Perancangan Skema Pengalamatan IP dan Routing

3.2.1 Proses Kalkulasi VLSM

Address Space yang digunakan: 192.168.100.0/24. Metode VLSM diterapkan dengan mengalokasikan subnet dari yang membutuhkan host terbanyak ke yang paling sedikit. Desain ip address dgn vlsm (Internet Engineering Task Force, 1993).

Langkah-langkah kalkulasi:

1. Identifikasi kebutuhan host per subnet:
 - Subnet A (Client Network): 10 host (klien + gateway router)
 - Subnet C (Backend Network): 3 host (HAProxy backend + WS1 + WS2)
 - Subnet B (Frontend/LB Network): 2 host (Router R1 eth1 + HAProxy eth0)
2. Urutkan dari kebutuhan terbesar ke terkecil dan tentukan prefix:
 - 10 host → /28 ($2^4 = 16$ alamat, 14 usable)
 - 3 host → /29 ($2^3 = 8$ alamat, 6 usable)
 - 2 host → /30 ($2^2 = 4$ alamat, 2 usable)
3. Alokasikan secara berurutan dengan menjaga batas subnet yang benar.

3.2.2 Pembagian Subnet (VLSM)

Table 5 Pembagian Subnet menggunakan VLSM

No	Nama Subnet	Keb. Host	Network Address	Subnet Mask	CIDR	Broadcast	Host Pertama	Host Terakhir	Host Tersedia
1	Client Network	10	192.168.100.0	255.255.255.240	/28	192.168.100.15	192.168.100.1	192.168.100.14	14
2	Backend Network	3	192.168.100.16	255.255.255.248	/29	192.168.100.23	192.168.100.17	192.168.100.22	6
3	Frontend/LB Net	2	192.168.100.24	255.255.255.52	/30	192.168.100.27	192.168.100.25	192.168.100.26	2

3.2.3 2 IP Address Plan per Perangkat

Table 6 IP Address Plan per Perangkat

No	Perangkat	Interface	IP Address	Subnet Mask	Gateway	Keterangan
1	Router R1	eth0	192.168.100.1	255.255.255.240	—	Gateway Client Network (/28)
2	Router R1	eth1	192.168.100.25	255.255.255.252	—	Gateway Frontend/LB Net (/30)
3	HAProxy LB	eth0	192.168.100.26	255.255.255.252	192.168.100.25	Frontend, terhubung ke Router R1
4	HAProxy LB	eth1	192.168.100.17	255.255.255.248	—	Backend, gateway untuk WS1 & WS2
5	Web Server 1	eth0	192.168.100.18	255.255.255.248	192.168.100.17	Backend Network (/29)
6	Web Server 2	eth0	192.168.100.19	255.255.255.248	192.168.100.17	Backend Network (/29)
7	Client PC1	eth0	192.168.100.2	255.255.255.240	192.168.100.1	Client Network (/28)

3.2.4 Routing Table (Static Routes)

Table 7 Routing Table pada router 1

Destination	Mask	Next Hop	Interface	Keterangan
192.168.100.0	255.255.255.240	-	eth0	Client Network (directly connected)
192.168.100.24	255.255.255.252	-	eth1	Frontend/LB Net (directly connected)
192.168.100.16	255.255.255.248	192.168.100.26	eth1	Backend Network via HAProxy
0.0.0.0	0.0.0.0	NAT/Cloud	eth0	Default route ke Internet

3.3 Alur Komunikasi Sistem

Berikut adalah penjelasan alur komunikasi lengkap pada sistem High Availability yang dirancang: Klien (PC1, IP: 192.168.100.2) mengirimkan request HTTP ke virtual IP HAProxy. Request diteruskan melalui Router R1 (192.168.100.1) yang berfungsi sebagai gateway.

1. Router R1 meneruskan paket ke HAProxy Load Balancer (192.168.100.26) melalui frontend network (/30). Routing statis memastikan paket mencapai tujuan yang tepat.
2. HAProxy menerima request pada port 80 (frontend listener) dan memilih server backend menggunakan algoritma Round Robin. Pada iterasi ganjil, request diteruskan ke WS1 (192.168.100.18); pada iterasi genap, ke WS2 (192.168.100.19).
3. Server backend yang dipilih memproses request dan mengembalikan respons HTTP ke HAProxy, yang kemudian meneruskannya kembali ke klien.
4. Secara paralel, HAProxy menjalankan health check setiap 2 detik ke masing-masing backend. Jika 3 health check berturut-turut gagal (parameter: fail=3), server tersebut ditandai sebagai DOWN dan semua trafik baru dialihkan ke server yang masih aktif.

BAB IV

RENCANA IMPLEMENTASI

4.1 Persiapan Lingkungan Implementasi

Tahap implementasi diawali dengan menyiapkan seluruh lingkungan kerja yang akan digunakan dalam pembangunan sistem High Availability Server. Implementasi dilakukan pada komputer host yang menjalankan sistem operasi Windows dengan memanfaatkan VirtualBox sebagai hypervisor untuk menjalankan beberapa mesin virtual Ubuntu Server serta GNS3 sebagai simulator jaringan.

Perangkat lunak yang digunakan meliputi VirtualBox versi 7.x, Ubuntu Server 22.04 LTS, GNS3 versi 2.2.x, HAProxy sebagai load balancer, Apache2 sebagai web server, serta Cisco IOS Router yang dijalankan melalui GNS3. Seluruh perangkat lunak tersebut dipilih karena bersifat stabil, banyak digunakan pada lingkungan industri, serta mendukung implementasi High Availability secara optimal.

Sebelum konfigurasi dilakukan, setiap virtual machine dipastikan dapat berjalan dengan baik dan memiliki spesifikasi perangkat keras virtual yang memadai, seperti RAM, kapasitas penyimpanan, serta jumlah adapter jaringan sesuai kebutuhan topologi yang telah dirancang pada Bab III.

4.2 Pembuatan Virtual Machine Ubuntu Server

Implementasi dimulai dengan membuat tiga buah Virtual Machine menggunakan Oracle VirtualBox. Ketiga virtual machine tersebut masing-masing memiliki fungsi yang berbeda, yaitu sebagai server load balancer dan dua buah web server backend. Adapun pembagian fungsi virtual machine adalah sebagai berikut.

Table 8 Fungsi virtual machine yang digunakan.

No	Nama Virtual Machine	Fungsi
1	HAProxy	Load Balancer
2	Web Server 1	Backend Server 1
3	Web Server 2	Backend Server 2

Seluruh virtual machine menggunakan sistem operasi Ubuntu Server 22.04 LTS agar penggunaan sumber daya lebih ringan dibandingkan sistem operasi berbasis desktop. Setelah proses instalasi selesai, masing-masing virtual machine diberikan hostname sesuai fungsinya untuk mempermudah proses administrasi sistem.

Tahapan berikutnya adalah melakukan pembaruan (update) repository sistem agar seluruh paket yang akan diinstal menggunakan versi terbaru.

```
sudo apt update
```

```
sudo apt upgrade -y
```

Perintah tersebut dijalankan pada seluruh virtual machine sehingga setiap server memiliki kondisi sistem operasi yang sama sebelum dilakukan konfigurasi lebih lanjut.

4.3 Konfigurasi Jaringan Virtual Machine

Setelah seluruh virtual machine berhasil dibuat, tahap berikutnya adalah melakukan konfigurasi jaringan sesuai dengan rancangan IP Address Plan yang telah disusun pada Bab III. HAProxy dikonfigurasi menggunakan dua buah network interface. Interface pertama digunakan sebagai frontend yang menerima permintaan dari client melalui router, sedangkan interface kedua digunakan sebagai backend yang terhubung langsung dengan kedua web server. Sementara itu, masing-masing web server hanya menggunakan satu interface jaringan yang terhubung pada jaringan backend.

```
timeout client 50000
timeout server 50000
errorfile 400 /etc/haproxy/errors/400.http
errorfile 403 /etc/haproxy/errors/403.http
errorfile 408 /etc/haproxy/errors/408.http
errorfile 500 /etc/haproxy/errors/500.http
errorfile 502 /etc/haproxy/errors/502.http
errorfile 503 /etc/haproxy/errors/503.http
errorfile 504 /etc/haproxy/errors/504.http

frontend web_frontend
    bind *:80
    default_backend web_servers

backend web_servers
    balance roundrobin
    option httpchk GET /
    server webserver1 192.168.56.11:80 check
    server webserver2 192.168.56.12:80 check

adib@haproxy:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:06:63:26 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe06:6326/64 scope link
        valid_lft forever preferred_lft forever
adib@haproxy:~$
```

Gambar 4 Konfigurasi IP Address pada HAProxy

Konfigurasi alamat IP dilakukan secara statis sehingga setiap perangkat selalu memperoleh alamat yang sama setiap kali sistem dijalankan. Pendekatan ini dipilih untuk mempermudah proses routing serta konfigurasi HAProxy.

```

System information as of Fri Jun 26 12:39:31 PM UTC 2026

System load:  0.01      Processes:           99
Usage of /:   34.1% of 9.74GB  Users logged in:    0
Memory usage: 22%      IPv4 address for enp0s3: 192.168.56.11
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

238 updates can be applied immediately.
178 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection
or proxy settings

Last login: Fri Jun 26 12:02:37 UTC 2026 on tty1
adib@webserver1:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b8:3b:c2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.11/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:feb8:3bc2/64 scope link
        valid_lft forever preferred_lft forever
adib@webserver1:~$

```

Gambar 5 Konfigurasi alamat IP pada Web Server 1

```

System information as of Fri Jun 26 12:41:34 PM UTC 2026

System load:  0.08      Processes:           100
Usage of /:   34.1% of 9.74GB  Users logged in:    0
Memory usage: 22%      IPv4 address for enp0s3: 192.168.56.12
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

238 updates can be applied immediately.
178 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection
or proxy settings

Last login: Fri Jun 26 08:26:04 UTC 2026 on tty1
adib@webserver2:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b2:60:17 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.12/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:feb2:6017/64 scope link
        valid_lft forever preferred_lft forever
adib@webserver2:~$

```

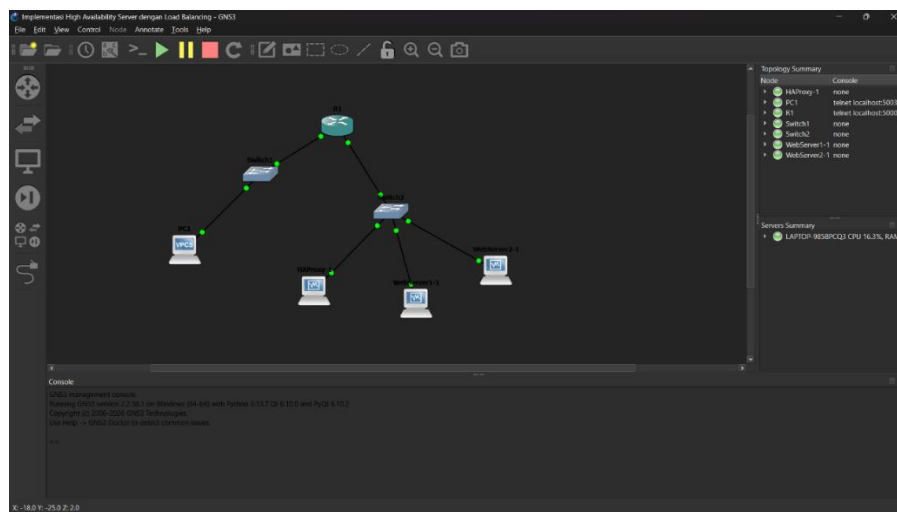
Gambar 6 Konfigurasi alamat IP pada Web Server 2

Setelah konfigurasi selesai dilakukan, konektivitas antar perangkat diuji menggunakan utilitas ping untuk memastikan seluruh perangkat telah berada pada subnet yang benar dan dapat saling berkomunikasi. Hasil pengujian menunjukkan bahwa seluruh virtual machine berhasil saling terhubung tanpa mengalami packet loss sehingga implementasi dapat dilanjutkan ke tahap berikutnya.

4.4 Integrasi Virtual Machine dengan GNS3

Setelah konfigurasi jaringan pada virtual machine selesai dilakukan, seluruh virtual machine kemudian diintegrasikan ke dalam lingkungan simulasi GNS3. Integrasi dilakukan menggunakan VirtualBox Guest sehingga setiap virtual machine dapat berfungsi sebagai node pada topologi jaringan yang telah dirancang sebelumnya.

Pada tahap ini juga ditambahkan beberapa perangkat jaringan lain seperti Cisco Router, Ethernet Switch, Cloud NAT, dan VPCS sebagai client. Seluruh perangkat kemudian dihubungkan menggunakan koneksi Ethernet sesuai dengan desain topologi sehingga terbentuk jalur komunikasi antara client, router, HAProxy, serta kedua web server backend.



Gambar 7 Topologi implementasi jaringan pada GNS3

Setelah seluruh koneksi selesai dibuat, dilakukan pengujian awal menggunakan perintah ping dari client menuju router, HAProxy, Web Server 1, dan Web Server 2. Seluruh perangkat berhasil memberikan balasan (reply), yang menunjukkan bahwa konfigurasi routing dasar telah berjalan dengan baik.

4.5 Hasil Implementasi Tahap Awal

Berdasarkan tahapan implementasi yang telah dilakukan, seluruh lingkungan kerja berhasil dipersiapkan sesuai dengan rancangan sistem pada Bab III. Tiga buah virtual machine Ubuntu Server berhasil dibuat menggunakan Oracle VirtualBox dengan pembagian fungsi sebagai Load Balancer (HAProxy), Web Server 1, dan Web Server 2. Seluruh virtual machine dapat dijalankan dengan baik tanpa mengalami kendala yang berarti selama proses instalasi.

```
PC1> trace 192.168.56.10
Trace to 192.168.56.10, 8 hops max, press Ctrl+C to stop
 1  192.168.10.1    10.279 ms  10.178 ms  10.180 ms
 2  *192.168.56.10  20.362 ms (ICMP type:3, code:3, Destination port unreachable)
```

Gambar 8 Hasil traceroute dari PC1 menuju HAProxy

Konfigurasi alamat IP statis pada masing-masing virtual machine juga telah berhasil diterapkan sesuai dengan IP Address Plan yang telah dirancang sebelumnya. Penggunaan alamat IP statis memberikan kemudahan dalam proses administrasi jaringan serta memastikan setiap perangkat memiliki identitas yang tetap selama proses implementasi berlangsung.

Tahap berikutnya yaitu integrasi virtual machine ke dalam lingkungan simulasi GNS3 berhasil dilakukan. Seluruh perangkat jaringan, meliputi Router Cisco, Ethernet Switch, Cloud NAT, HAProxy, kedua Web Server, serta VPCS sebagai client, telah terhubung sesuai dengan topologi yang dirancang.

```
C    192.168.10.0/24 is directly connected, FastEthernet0/0
C    192.168.56.0/24 is directly connected, FastEthernet1/0
R1#
```

Gambar 9 Routing table pada Router R1

Pengujian konektivitas menggunakan perintah ping menunjukkan bahwa seluruh perangkat dapat saling berkomunikasi dengan baik tanpa mengalami packet loss. Hal tersebut menunjukkan bahwa konfigurasi jaringan dasar, pengalamatan IP, serta routing awal telah berjalan sesuai dengan perencanaan sehingga sistem siap untuk memasuki tahap implementasi layanan pada progres berikutnya.

```
Build time: Apr 10 2019 02:42:20
Copyright (c) 2007-2014, Paul Meng (mirnshi@gmail.com)
All rights reserved.

VPCS is free software, distributed under the terms of the "BSD" licence.
Source code and license can be found at vpcs.sf.net.
For more information, please visit wiki.freecode.com.cn.

Press '?' to get help.

Executing the startup file

PC1> ip 192.168.10.2 255.255.255.0 192.168.10.1
Checking for duplicate address...
PC1 : 192.168.10.2 255.255.255.0 gateway 192.168.10.1

PC1> ping 192.168.10.1
84 bytes from 192.168.10.1 icmp_seq=1 ttl=255 time=10.244 ms
84 bytes from 192.168.10.1 icmp_seq=2 ttl=255 time=6.610 ms
84 bytes from 192.168.10.1 icmp_seq=3 ttl=255 time=6.704 ms
84 bytes from 192.168.10.1 icmp_seq=4 ttl=255 time=5.600 ms
84 bytes from 192.168.10.1 icmp_seq=5 ttl=255 time=5.597 ms

PC1> ping 192.168.56.10
84 bytes from 192.168.56.10 icmp_seq=1 ttl=63 time=18.055 ms
84 bytes from 192.168.56.10 icmp_seq=2 ttl=63 time=16.360 ms
84 bytes from 192.168.56.10 icmp_seq=3 ttl=63 time=16.526 ms
84 bytes from 192.168.56.10 icmp_seq=4 ttl=63 time=16.532 ms
84 bytes from 192.168.56.10 icmp_seq=5 ttl=63 time=17.998 ms

PC1> ping 192.168.56.11
84 bytes from 192.168.56.11 icmp_seq=1 ttl=63 time=20.065 ms
84 bytes from 192.168.56.11 icmp_seq=2 ttl=63 time=16.485 ms
84 bytes from 192.168.56.11 icmp_seq=3 ttl=63 time=17.523 ms
84 bytes from 192.168.56.11 icmp_seq=4 ttl=63 time=16.900 ms
84 bytes from 192.168.56.11 icmp_seq=5 ttl=63 time=17.079 ms

PC1> ping 192.168.56.12
84 bytes from 192.168.56.12 icmp_seq=1 ttl=63 time=17.797 ms
84 bytes from 192.168.56.12 icmp_seq=2 ttl=63 time=15.235 ms
84 bytes from 192.168.56.12 icmp_seq=3 ttl=63 time=16.524 ms
84 bytes from 192.168.56.12 icmp_seq=4 ttl=63 time=17.222 ms
84 bytes from 192.168.56.12 icmp_seq=5 ttl=63 time=15.951 ms

PC1> curl http://192.168.56.10
Bad command: "curl http://192.168.56.10". Use ? for help.

PC1>
```

Gambar 10 Pengujian konektivitas dari PC1 ke seluruh perangkat

4.6 Instalasi dan Konfigurasi Apache2

Setelah infrastruktur jaringan berhasil dibangun pada tahap sebelumnya, implementasi dilanjutkan dengan memasang layanan web server pada masing-masing backend server. Layanan yang digunakan adalah Apache HTTP Server (Apache2) karena merupakan salah satu web server yang bersifat open source, ringan, stabil, dan banyak digunakan pada implementasi layanan berbasis Linux.

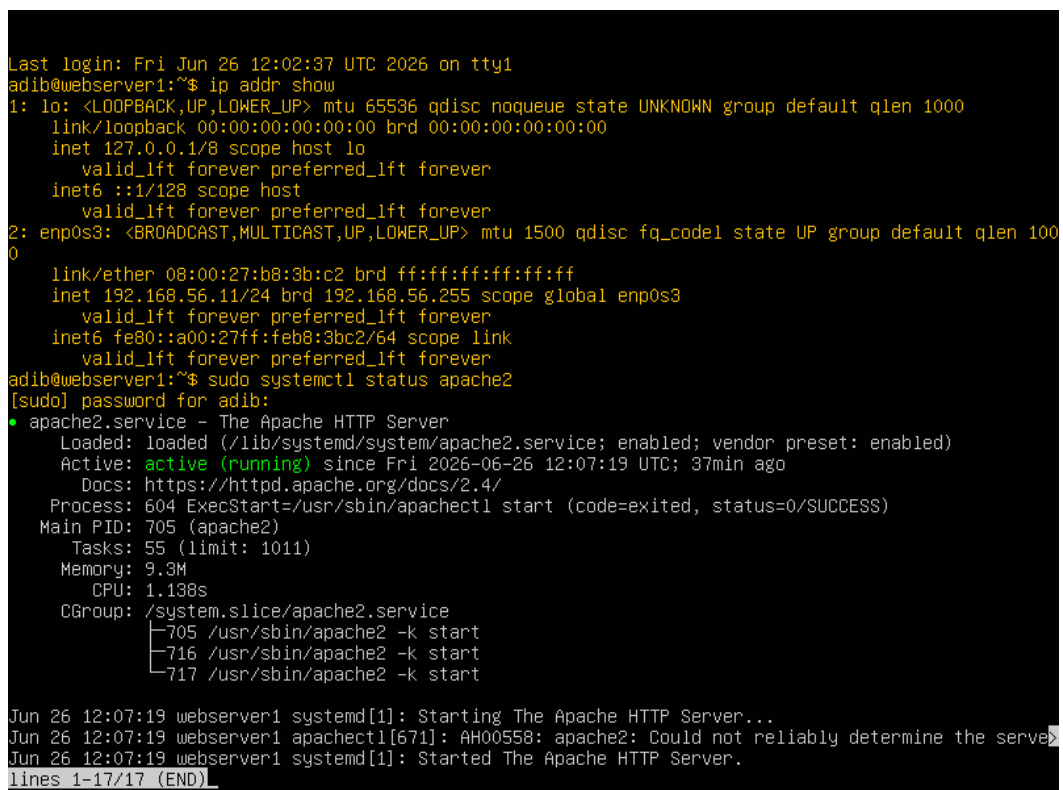
Instalasi Apache2 dilakukan pada Web Server 1 dan Web Server 2 menggunakan manajer paket APT pada sistem operasi Ubuntu Server. Setelah proses instalasi selesai, layanan Apache dijalankan serta diatur agar aktif secara otomatis ketika sistem dinyalakan

kembali (*boot*). Untuk memastikan layanan berjalan dengan baik, dilakukan pemeriksaan status menggunakan perintah berikut.

```
sudo systemctl status apache2
```

Hasil pemeriksaan menunjukkan bahwa layanan Apache2 pada kedua web server berada pada status *active (running)*. Hal ini menandakan bahwa web server telah berhasil diinstal dan siap menerima permintaan HTTP dari HAProxy.

Selanjutnya, masing-masing web server diberikan halaman web sederhana yang berbeda sebagai identitas server sehingga proses distribusi trafik nantinya dapat diamati dengan mudah. Web Server 1 menampilkan identitas "WEB SERVER 1", sedangkan Web Server 2 menampilkan identitas "WEB SERVER 2".



```
Last login: Fri Jun 26 12:02:37 UTC 2026 on tty1
adib@webserver1:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b8:3b:c2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.11/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:feb8:3bc2/64 scope link
        valid_lft forever preferred_lft forever
adib@webserver1:~$ sudo systemctl status apache2
[sudo] password for adib:
• apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-06-26 12:07:19 UTC; 37min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 604 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 705 (apache2)
    Tasks: 55 (limit: 1011)
   Memory: 9.3M
      CPU: 1.138s
   CGroup: /system.slice/apache2.service
           └─705 /usr/sbin/apache2 -k start
             └─716 /usr/sbin/apache2 -k start
               └─717 /usr/sbin/apache2 -k start

Jun 26 12:07:19 webserver1 systemd[1]: Starting The Apache HTTP Server...
Jun 26 12:07:19 webserver1 apachectl[671]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, please see the /etc/httpd/conf/httpd.conf file for instructions on how to set the servername.
Jun 26 12:07:19 webserver1 systemd[1]: Started The Apache HTTP Server.
lines 1-17/17 (END)
```

Gambar 11 Status layanan Apache2 pada Web Server 1

```

Last login: Fri Jun 26 08:26:04 UTC 2026 on tty1
adib@webserver2:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b2:60:17 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.12/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:feb2:6017/64 scope link
        valid_lft forever preferred_lft forever
adib@webserver2:~$ sudo systemctl status apache2
[sudo] password for adib:
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-06-26 09:06:38 UTC; 3h 40min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 605 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
    Main PID: 833 (apache2)
       Tasks: 55 (limit: 1011)
      Memory: 9.8M
         CPU: 6.576s
    CGroup: /system.slice/apache2.service
            └─833 /usr/sbin/apache2 -k start
              └─835 /usr/sbin/apache2 -k start
                └─836 /usr/sbin/apache2 -k start

Jun 26 09:06:28 webserver2 systemd[1]: Starting The Apache HTTP Server...
Jun 26 09:06:38 webserver2 apachectl[665]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, please add the 'ServerName' directive to the configuration to avoid this warning.
Jun 26 09:06:38 webserver2 systemd[1]: Started The Apache HTTP Server.
lines 1-17/17 (END)

```

Gambar 12 Status layanan Apache2 pada Web Server 2

4.7 Konfigurasi HAProxy

Tahap berikutnya adalah melakukan konfigurasi HAProxy sebagai load balancer yang bertugas mendistribusikan permintaan dari client menuju kedua backend server. Konfigurasi dilakukan dengan mengubah file `/etc/haproxy/haproxy.cfg`. Pada implementasi ini digunakan metode Round Robin, sehingga setiap permintaan HTTP akan dialihkan secara bergantian ke Web Server 1 dan Web Server 2.

```

GNU nano 6.2 /etc/haproxy/haproxy.cfg
# See: https://ssl-config.mozilla.org/#server=haproxy&server-version=2.0.3&config=intermedi
ssl-default-bind-ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-EC
ssl-default-bind-ciphersuites TLS_AES_128_GCM_SHA256:TLS_AES_256_GCM_SHA384:TLS_CHACHA20_PQ
ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets

defaults
    log global
    mode http
    option httplog
    option dontlognull
    timeout connect 5000
    timeout client 50000
    timeout server 50000
    errorfile 400 /etc/haproxy/errors/400.http
    errorfile 403 /etc/haproxy/errors/403.http
    errorfile 408 /etc/haproxy/errors/408.http
    errorfile 500 /etc/haproxy/errors/500.http
    errorfile 502 /etc/haproxy/errors/502.http
    errorfile 503 /etc/haproxy/errors/503.http
    errorfile 504 /etc/haproxy/errors/504.http

frontend web_front
    bind *:80
    default_backend web_servers

backend web_servers
    balance roundrobin
    option httpchk GET /
    server webserver1 192.168.56.11:80 check
    server webserver2 192.168.56.12:80 check

```

Gambar 13 Konfigurasi file haproxy.cfg

Selain itu, konfigurasi juga memanfaatkan fitur HTTP Health Check menggunakan parameter option `httpchk GET /`. Fitur ini memungkinkan HAProxy melakukan pemeriksaan secara berkala terhadap kondisi masing-masing backend server. Apabila salah satu server tidak memberikan respons, maka server tersebut secara otomatis dikeluarkan dari daftar server aktif sehingga seluruh permintaan akan dialihkan ke server yang masih berjalan.

```
adib@haproxy:~$ sudo systemctl status haproxy
[sudo] password for adib:
• haproxy.service - HAProxy Load Balancer
   Loaded: loaded (/lib/systemd/system/haproxy.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-06-26 09:06:08 UTC; 3h 1min ago
     Docs: man:haproxy(1)
           file:/usr/share/doc/haproxy/configuration.txt.gz
   Process: 652 ExecStartPre=/usr/sbin/haproxy -Ws -f $CONFIG -c -q $EXTRA_OPTS (code=exited, status=0/SUCCESS)
   Main PID: 683 (haproxy)
      Tasks: 2 (limit: 1011)
     Memory: 74.7M
        CPU: 5.642s
    CGroup: /system.slice/haproxy.service
            └─683 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -S /run/haproxy.sock
              750 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -S /run/haproxy.sock

Jun 26 09:06:08 haproxy systemd[1]: Started HAProxy Load Balancer.
Jun 26 09:06:09 haproxy haproxy[750]: [WARNING] (750) : Server web_servers/webserver1 is DOWN, reason: connect() to 192.168.56.10:80 failed. (2/0)
Jun 26 09:06:09 haproxy haproxy[750]: [WARNING] (750) : Server web_servers/webserver2 is DOWN, reason: connect() to 192.168.56.11:80 failed. (2/0)
Jun 26 09:06:09 haproxy haproxy[750]: [NOTICE] (750) : haproxy version is 2.4.30-0ubuntu0.22.04.2
Jun 26 09:06:09 haproxy haproxy[750]: [NOTICE] (750) : path to executable is /usr/sbin/haproxy
Jun 26 09:06:09 haproxy haproxy[750]: [ALERT] (750) : backend 'web_servers' has no server available
Jun 26 09:06:39 haproxy haproxy[750]: [WARNING] (750) : Server web_servers/webserver1 is UP, reason: connect() to 192.168.56.10:80 succeeded. (0/0)
Jun 26 09:06:49 haproxy haproxy[750]: [WARNING] (750) : Server web_servers/webserver2 is UP, reason: connect() to 192.168.56.11:80 succeeded. (0/0)
Jun 26 12:02:59 haproxy haproxy[750]: [WARNING] (750) : Server web_servers/webserver1 is DOWN, reason: connect() to 192.168.56.10:80 failed. (2/0)
Jun 26 12:07:22 haproxy haproxy[750]: [WARNING] (750) : Server web_servers/webserver1 is UP, reason: connect() to 192.168.56.10:80 succeeded. (0/0)
lines 1-24/24 (END)
```

Gambar 14 Status layanan HAProxy beserta health check backend

Setelah konfigurasi selesai dilakukan, layanan HAProxy dijalankan kembali menggunakan perintah:

```
sudo systemctl restart haproxy
```

Selanjutnya dilakukan pemeriksaan status layanan menggunakan:

```
sudo systemctl status haproxy
```

Hasil pengujian menunjukkan bahwa HAProxy berhasil berjalan dengan status active (running) dan mampu mendeteksi kondisi kedua backend server.

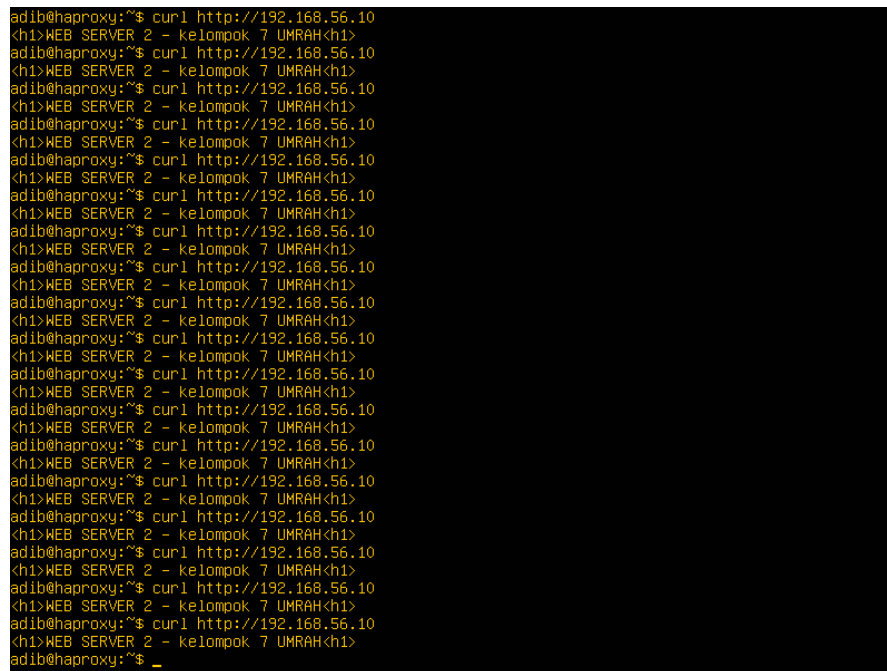
4.8 Pengujian Load Balancing

Setelah konfigurasi HAProxy selesai dilakukan, tahap berikutnya adalah menguji mekanisme distribusi trafik menggunakan algoritma Round Robin. Pengujian dilakukan dengan mengirimkan permintaan HTTP secara berulang menggunakan perintah curl menuju alamat IP HAProxy.

```
curl http://192.168.56.10
```

Hasil pengujian menunjukkan bahwa respons yang diterima client berasal secara bergantian dari Web Server 1 dan Web Server 2. Pergantian respons tersebut menunjukkan

bahwa HAProxy telah berhasil menerapkan algoritma Round Robin sesuai konfigurasi yang telah dibuat.

A screenshot of a terminal window with a black background and yellow text. It shows a series of 20 curl commands being executed from a shell prompt 'adib@haproxy:~\$'. Each command is 'curl http://192.168.56.10'. The output of each command is a line starting with '<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>'. The output is identical for all 20 requests, demonstrating that all traffic is being routed to the same backend server (group 7) in this specific test run.

```
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$
```

Gambar 15 Hasil pengujian Round Robin menggunakan perintah curl.

Implementasi ini membuktikan bahwa distribusi beban kerja tidak hanya bergantung pada satu server, tetapi dibagi secara merata kepada kedua backend server sehingga pemanfaatan sumber daya menjadi lebih optimal.

4.9 Pengujian Failover

Selain melakukan pengujian distribusi trafik, sistem juga diuji terhadap kemampuan failover. Pengujian dilakukan dengan menghentikan layanan Apache pada salah satu backend server sehingga server tersebut dianggap tidak aktif oleh HAProxy. Ketika salah satu backend server berhenti memberikan layanan, HAProxy secara otomatis mendeteksi kondisi tersebut melalui mekanisme health check dan menghapus server yang bermasalah dari daftar server aktif.

Selanjutnya dilakukan kembali pengujian menggunakan perintah curl menuju alamat IP HAProxy. Hasil pengujian menunjukkan bahwa seluruh permintaan dialihkan menuju backend server yang masih aktif tanpa memerlukan perubahan konfigurasi maupun intervensi administrator.

```

System load: 0.0          Processes:          100
Usage of /: 33.1% of 9.74GB Users logged in:    0
Memory usage: 29%        IPv4 address for enp0s3: 192.168.56.10
Swap usage: 0%

Expanded Security Maintenance for Applications is not enabled.

238 updates can be applied immediately.
178 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection
or proxy settings

Last login: Fri Jun 26 08:25:43 UTC 2026 on tty1
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 1 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 1 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 1 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 1 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMRAH<h1>
adib@haproxy:~$ _

```

Gambar 16 Hasil pengujian failover ketika salah satu backend server tidak aktif.

Hasil tersebut menunjukkan bahwa mekanisme High Availability telah berhasil diterapkan. Layanan tetap dapat diakses meskipun salah satu web server mengalami gangguan sehingga kontinuitas layanan tetap terjaga.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan implementasi lanjutan yang telah dilakukan pada Laporan Kemajuan 3, sistem High Availability Server berhasil dikembangkan dengan menambahkan layanan Apache2 pada kedua backend server serta HAProxy sebagai load balancer. Konfigurasi yang diterapkan mampu mendistribusikan permintaan client menggunakan algoritma Round Robin secara bergantian ke kedua web server.

Selain itu, mekanisme health check pada HAProxy berhasil mendeteksi kondisi backend server secara otomatis. Ketika salah satu web server mengalami gangguan, seluruh permintaan client tetap dapat dilayani oleh server lain yang masih aktif. Hasil tersebut menunjukkan bahwa tujuan implementasi High Availability telah tercapai, yaitu meningkatkan ketersediaan layanan dan meminimalkan gangguan terhadap pengguna.

5.2 Rencana Tahap Selanjutnya

Pada tahap akhir proyek akan dilakukan penyempurnaan dokumentasi implementasi, analisis hasil pengujian secara lebih mendalam, evaluasi performa sistem, serta penyusunan laporan akhir yang mencakup seluruh tahapan mulai dari perencanaan, implementasi, hingga pengujian sistem High Availability Server menggunakan HAProxy dan dua backend web server.

DAFTAR PUSTAKA

- Canonical Ltd. (2024). *Ubuntu Server 22.04 LTS Documentation*. Retrieved from Ubuntu:
<https://ubuntu.com/server/docs>
- Forouzan, B. A. (2013). *Data Communication and Networking*. New York: McGraw-Hill Education.
- GNS3 Technologies. (2024). *GNS3 Documentation*. Retrieved from GNS3:
<https://docs.gns3.com>
- HAProxy Technologies. (2024). *HAProxy Configuration Manual Version 2.4*. Retrieved from HAProxy: <https://www.haproxy.org/download/2.4/doc/configuration.txt>
- Internet Engineering Task Force. (1981). *RFC 791 — Internet Protocol*. Retrieved from IETF Datatracker: <https://datatracker.ietf.org/doc/html/rfc791>
- Internet Engineering Task Force. (1993). *RFC 1519 — Classless Inter-Domain Routing (CIDR)*. Retrieved from IETF Datatracker: <https://datatracker.ietf.org/doc/html/rfc1519>
- Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks*. Boston: Person Education.